

Privacy-Preserving Smart Contracts Engine

Zhiyi Li, Tianpei Lu

Abstract—We present PPSC, a dynamic hybrid engine for confidential smart contracts. PPSC keeps contract code and public handles on a host chain. Persistent private values reside off chain as either threshold-FHE ciphertexts or authenticated secret sharings. A public, contract-defined operator sequence assigns each operator to FHE or MPC and explicitly specifies every FHE–MPC conversion. The active committee holds shares of the FHE secret key and uses them for conversion. An authorized Pick returns a qualified set of key shares to the owner, who reconstructs the key and decrypts locally. When the committee changes, it transfers the FHE key shares, live contract-value shares, MAC shares, and remaining conversion or preprocessing values to the incoming committee. We specify this design, the algebraic conditions for bidirectional conversion, and fluid handoff. PRSS, authenticated opening, scalar and packed degree reducing, and conversion preprocessing support the MPC operators. The concrete malicious-secure conversion and fluid handoff protocols remain to be completed; we make no performance claim before the prototype and evaluation are complete.

I. INTRODUCTION

Public smart-contract platforms expose contract code, calls, and state updates to validating nodes. Confidential-contract systems must preserve public contract execution and prevent validators and execution services from learning private values. FHEVM-style designs address the problem by keeping values encrypted and evaluating contract operators homomorphically. This approach supports packed arithmetic. Forcing every operator through FHE is costly or semantically awkward for exact comparisons, private selection, range checks, and data-dependent operations implemented as data-oblivious circuits.

Moving such operators to MPC creates a second problem: one contract state must cross between an FHE ciphertext and an authenticated sharing without exposing the logical value, the FHE decryption intermediate, or reusable mask state. Moreover, FHEVM-style architectures commonly assign threshold key generation and decryption to a dedicated KMS service [1]. Once FHE and MPC are executed by a committee, the same committee can hold shares of the threshold FHE key. Threshold key management remains part of the engine. The design removes the separate, long-lived KMS trust boundary. After an authorized Pick, the owner receives enough key shares to reconstruct the FHE key and decrypts the ciphertext locally.

We therefore propose PPSC, a dynamic hybrid engine in which a public, contract-defined operator sequence places each

operator in the FHE or MPC domain. Private values persist off chain as either threshold-FHE ciphertexts or authenticated secret sharings, and explicit conversion edges connect the two domains. The host chain stores the contract and public data references. The active committee holds the FHE secret-key shares, MPC value and MAC shares, and conversion or preprocessing state. When the committee rotates, these objects are handed to the incoming committee together.

This organization yields the paper’s causal chain. Hybrid execution requires bidirectional conversion. The conversion uses shares of the threshold FHE key. The active committee therefore holds these key shares together with live value shares, MAC shares, and preprocessing values. Committee rotation transfers all of these values to the incoming committee. PRSS, correlated pairs, DN reducing, packing, and information-theoretic MACs support MPC execution.

a) Contributions: First, we define FHE–MPC execution for confidential contracts. It has exactly two persistent private representations, FHE ciphertexts and authenticated MPC sharings. The contract states which operators use FHE, which use MPC, and where conversion occurs.

Second, the active committee holds the threshold FHE key shares together with live contract-value shares, MAC shares, and conversion or preprocessing values. These values follow the same committee handoff. This design eliminates a separate long-lived KMS committee. Threshold key operations remain necessary. Authorized key opening gives the owner enough shares to reconstruct the key.

Third, we specify bidirectional FHE–MPC conversion and fluid committee handoff. The conversion parameters distinguish the FHE level modulus and scale from the MPC field and state the rounding and wrap-around conditions. The handoff transfers the FHE key shares, live state, MAC shares, and remaining conversion or preprocessing state to the incoming committee.

b) Organization: Section I-A separates the system from prior private-contract, threshold-FHE, and dynamic-MPC constructions. Section I-B fixes the algebraic domains and sharing primitives. Section II presents the FHE–MPC operators, components, state, and dataflow. Section III specifies FHE–MPC conversion, fluid committee handoff, and the authenticated MPC protocols. Section IV records the security conditions, followed by discussion and ethics considerations.

A. Related Work

Prior work provides private ledger transitions, private contract execution, FHE-based encrypted state, threshold key operations, and dynamic MPC. The relevant integration problem is to combine public data identifiers with committee-held key

shares, off-chain SS/FHE storage, contract-defined operators, and state transfer across changing executor groups.

a) *Proof- and hardware-based private execution:* Ben-Sasson *et al.* introduced Zerocash, which uses commitments, serial numbers, and zk-SNARKs to hide the origin, destination, and amount of payments and preserve public verification [2]. Bowe *et al.* generalized proof-carrying ledger transitions through Zexe, whose decentralized private computation model hides offline computations and validates transactions succinctly [3]. Hawk separates a contract into public and private portions; a manager evaluates the private part and submits proofs and commitments that bind settlement to the contract semantics [4]. These systems establish strong public verifiability. Computation over private state held by several users still requires a party or protocol to obtain a consistent witness for the relevant state transition.

Trusted hardware TEE-based systems evaluate private logic inside a hardware-isolated environment and post outputs, roots, or attestations to a ledger. Kaptchuk *et al.* show that a ledger can supply persistent state and publication evidence to an otherwise stateless trusted device [5]. This line of work clarifies rollback resistance and state freshness. Confidentiality continues to depend on the trusted-hardware boundary and its attestation and side-channel assumptions.

b) *MPC- and FHE-based confidential contracts:* MPC-based systems replace a single trusted manager with parties that jointly evaluate a private function over secret-shared inputs. zkHawk replaces Hawk’s manager by MPC and avoids the cost of evaluating a complete zk-SNARK inside the MPC circuit [6]. V-zkHawk gives a UC-style variant using commitments, signatures, NIZKs, and broadcast assumptions [7]. Eagle combines outsourced MPC, Pedersen commitments, and zero-knowledge proofs to support long-running private contracts for clients that participate only during a short input phase [8]. These systems motivate distributed private execution. A changing executor set additionally requires explicit migration of the private state and its authentication material.

Homomorphic-encryption systems Zether uses additively homomorphic ElGamal encryption and zero-knowledge proofs to keep balances encrypted in an account-based smart-contract setting [9]. zkay introduces privacy types and transforms privacy-annotated contracts into public contracts enforced by encryption and NIZK proofs [10]. smartFHE uses FHE so that evaluators compute on encrypted inputs, and users prove input well-formedness [11]. Zama’s fhEVM uses coprocessors for FHE evaluation, threshold MPC in a dedicated KMS for key generation and decryption, Gateway contracts for orchestration, and Solidity libraries for encrypted types [1]. Our system retains public identifiers for encrypted values. The active committee stores the FHE secret key in secret-shared form and executes contract-defined operators across the two representations.

c) *Threshold decryption and dynamic participation:* Threshold FHE distributes decryption authority so that no single party holds the FHE secret key [12]. Noah’s Ark studies threshold decryption for LWE-based FHE and uses

noise flooding to hide the original ciphertext noise before a decryption-related value is opened [13]. Fluid secret sharing refreshes shares of an unchanged secret so that shares collected in different periods cannot be accumulated, provided that the corruption bound holds in every period and obsolete state is erased [14]. Fluid MPC formalizes MPC with dynamic participants and divides a long computation into committee epochs connected by state handoff [15]. PPSC combines these ingredients with FHEVM-style state references and off-chain ciphertext storage.

B. Preliminaries

a) *Notation:* We write vectors in bold and use $x \stackrel{\$}{\leftarrow} \mathcal{D}$ for sampling x from a distribution $\mathcal{D}$. Let λ be the security parameter and let $\mathcal{P} = \{P_1, \dots, P_n\}$ be the participating parties, of which at most t may be corrupted. For a contract variable, $\mathcal{F}_\tau$ denotes the finite set of values of logical type τ , including its shape, signed range, and fixed-point scale. The MPC protocols perform arithmetic in a prime field $\mathbb{F}_Q$ and embed each admissible integer coordinate by reduction modulo Q ; centered lifting is unambiguous only for declared values in $(-Q/2, Q/2)$. Thus $\mathcal{F}_\tau$ is an application domain. MPC arithmetic is performed over $\mathbb{F}_Q$. Every non-zero field element is invertible, and all Shamir evaluation and packing points are distinct. The notation $[x]^{(d,Q)}$ denotes a degree- d Shamir sharing over this field; it follows the degree-and-modulus style used in prior threshold-FHE work [13]. A ring-based realization may have the same underlying finite set. The interpolation and MAC arguments stated here cover the field-based realization.

b) *t-threshold secret sharing:* Secret sharing lets a group hold a secret without giving the complete value to any single party. A t -threshold scheme protects the secret against any group of at most t parties. Any $t+1$ parties can recover it. The following definition states these two requirements.

Shamir secret sharing. Let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_Q \setminus \{0\}$ be distinct public evaluation points. To share $x \in \mathbb{F}_Q$ with threshold t , sample $f_x(X) = x + a_1X + \dots + a_tX^t$, where $a_1, \dots, a_t \stackrel{\$}{\leftarrow} \mathbb{F}_Q$, and give party P_i the share $[x]_i^{(t,Q)} = f_x(\alpha_i)$. Any $t+1$ valid shares reconstruct $f_x(0) = x$ by Lagrange interpolation. Any set of at most t shares is statistically independent of x [17]. When the degree is clear, $[x]_Q$ abbreviates a sharing over the same field $\mathbb{F}_Q$.

Replicated secret sharing. Replicated sharing gives several parties a common additive component. Let $\mathcal{H}_t = \{A \subseteq [n] : |A| = n - t\}$. A replicated sharing of $x \in \mathbb{F}_Q$ consists of components $\{x_A\}_{A \in \mathcal{H}_t}$ sampled uniformly subject to

$$x = \sum_{A \in \mathcal{H}_t} x_A, \quad (1)$$

where party P_i holds x_A exactly when $i \in A$. Every set of $t+1$ parties collectively holds every component and can reconstruct the sum. A coalition of at most t parties misses at least one independently randomized component. This representation is used later to derive non-interactive random Shamir sharings and correlated preprocessing material.

Linearity. For the operations used by the Shamir and replicated-sharing instantiations, parties can locally derive shares of $ax+by$ from shares of x and y for public coefficients a and b .

Degree reducing. Let $[x]^{(t,Q)}$ and $[y]^{(t,Q)}$ be degree- t Shamir sharings. Local multiplication evaluates $h_z(X) = f_x(X)f_y(X)$, where $\deg h_z \leq 2t$ and $h_z(0) = xy$. A DN-style [?] reducing returns the product to degree t by consuming a one-time correlated pair $([\rho]^{(t,Q)}, [\rho]^{(2t,Q)})$ whose two components encode the same random $\rho \in \mathbb{F}_Q$ [19], [21]. The parties reconstruct the masked value $w = xy + \rho$ and output $[xy]^{(t,Q)} = w - [\rho]^{(t,Q)}$. The complete protocol and the generation of the correlated pair are given later.

Packed sharing. Packed Shamir sharing encodes several values in one polynomial [18]. Let $\beta_1, \dots, \beta_L \in \mathbb{F}_Q$ be distinct public packing points disjoint from the party points. A packed sharing of $\mathbf{x} = (x_1, \dots, x_L)$ is represented by a polynomial $F_{\mathbf{x}}$ satisfying $F_{\mathbf{x}}(\beta_j) = x_j$ for every $j \in [L]$. Party P_i stores $F_{\mathbf{x}}(\alpha_i)$. A t -private baseline uses degree $D = t + L - 1$. Addition and public scalar multiplication are slot-wise. Multiplication doubles the polynomial degree and therefore requires packed degree reducing. The term *packed* denotes packed secret sharing in the MPC domain. Ciphertext-slot packing uses separate notation.

c) Authentication: Authentication lets the parties detect inconsistent shares and an incorrect opening. After values enter the arithmetic computation, PPSC uses an information-theoretic MAC over $\mathbb{F}_Q$. Let $\alpha \in \mathbb{F}_Q$ be a global secret MAC key that is itself secret-shared. An authenticated sharing of $x \in \mathbb{F}_Q$ is

$$\langle\langle x \rangle\rangle_Q = ([x]_Q, [\gamma_x]_Q), \quad \gamma_x = \alpha x. \quad (2)$$

Linearity gives authenticated sharings of public linear combinations without interaction. After reconstructing an intended opening x , the parties open $[\gamma_x]_Q - x[\alpha]_Q$ and accept only if the result is zero. A batch uses a fresh public random linear combination chosen after the opened values are fixed. This authentication method is used by active-secure arithmetic MPC, including SPDZ-style protocols [20], [21]; the concrete sharing and opening subprotocol must still provide consistent reconstruction.

d) Threshold FHE: PPSC uses threshold FHE for approximate packed arithmetic. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and let $\mathcal{R}_{q_\ell} = \mathcal{R}/q_\ell \mathcal{R}$ be the ciphertext ring at level ℓ , with public scale Δ_ℓ . A logical real value is represented by a bounded fixed-point integer vector $\boldsymbol{\mu}$, with public scale S and application value $\boldsymbol{\mu}/S$. At a conversion point, the operator sequence fixes the level, scale, layout, range, and admissible total decoding error. The secret key s is shared in the ring or module required by the selected threshold-FHE scheme. Committee co-management leaves this ring or module representation unchanged.

The conversion parameters select a level whose working coefficient modulus is a prime q_c and sets the MPC modulus $Q = q_c$, normally with $Q \equiv 1 \pmod{2N}$. This equality identifies coefficient residues only: FHE encoding and decoding

still map between packed slots and R_Q . The MPC protocols authenticate coefficients in $\mathbb{F}_Q$. It does not identify a scheme plaintext space with the logical domain $\mathcal{F}_\tau$. In particular, these parameters assume no separate plaintext modulus p and do not use $\Delta = \lfloor q/p \rfloor$.

e) Fluid security and dynamic committees: With a static committee, the same parties retain their shares throughout the execution. In a dynamic system, an adversary could compromise different parties in different epochs and accumulate obsolete shares. A fluid handoff refreshes the sharing when the active committee changes and preserves the underlying secret. Privacy therefore relies on the corruption bound holding in every epoch and on outgoing parties erasing obsolete shares and handoff randomness. The concrete handoff protocol is introduced only after this distinction is fixed.

f) Preprocessing: The protocols use an offline preprocessing phase. Before private inputs are available, the parties prepare data-independent random sharings and correlated randomness required by the online phase. Scalar degree reducing consumes a degree- t /degree- $2t$ sharing pair of one random field element; packed reducing consumes the analogous pair for one random vector. Each pair is used at most once. FHE-MPC conversion uses separately generated correlated randomness, as specified in Section III-B.

C. Threat Model

LU The adversary may read every public record, corrupt storage providers, and corrupt at most t parties in an active committee of size n . The corrupted set may change between epochs, which motivates fluid handoff.

The system has exactly two persistent confidential representations, SS and FHE. The contract fixes an operator sequence; no mode is assigned to an entire execution. Arithmetic SS protocols use the prime field fixed in Section I-B. FHE conversion requires compatible level, scale, range, layout, and error bounds.

Opening an SS value requires its authentication check. Opening an FHE output requires an authorized Pick of its key reference, reconstruction of the key, and local decryption by the owner. Offline preprocessing generates correlated randomness independently of user upload, and each such value is used only once. A handoff transfers the key, live values, MAC shares, and remaining conversion or preprocessing state to the incoming committee.

II. SYSTEM OVERVIEW

PPSC is a confidential smart-contract engine. [overview](#) Fig. 1

A. System Functions

a) Deploy: The user publishes a privacy contract containing its callable functions, confidential-variable declarations, and operator sequence. Deployment returns a public contract address. It does not upload the inputs of a later call.

b) *Upload and register*: Before invocation, the data owner chooses one of two persistent representations. For an SS value, the client creates shares and sends party P_i in the current execution committee $\mathcal{S}^\ell$ only its encrypted share $[x]_i$. For an FHE value, the client encrypts the value and sends the ciphertext to the designated storage nodes. The client then registers a public reference under a fresh dataId.

c) *Invoke*: The user calls a contract address and function selector with existing data identifiers. No plaintext, secret share, or FHE secret key is placed in the transaction. The host chain emits an invocation event. The off-chain runtime reads the referenced records and asks the selected committee to execute the operators defined by the contract.

d) *Pick and open*: To obtain an SS result, its owner posts a signed Pick that names the result identifier. To decrypt an FHE result under key k_b , the owner posts a Pick for dataId $_{k_b}$. The listed committee verifies the owner and returns a qualified set of key shares. The owner reconstructs k_b and decrypts the ciphertext locally.

B. System Components

The host chain stores the public contract and references confidential values by data identifier. Threshold-FHE ciphertexts and authenticated MPC sharings remain off chain. The contract specifies which operators use FHE, which operators use MPC, and where conversion occurs. The active committee holds the FHE secret-key shares together with the MPC shares. When the committee changes, these shares are transferred to the incoming committee.

The blocks in Fig. 1 have the following responsibilities.

a) *User and Client SDK/Relay*: The user supplies the contract, confidential inputs, invocation parameters, and signed Pick requests. The SDK prepares shares or an FHE ciphertext, delivers each share or ciphertext to its designated node, registers the public references, and submits Invoke and Pick transactions. It returns the opened result to the owner.

b) *Host chain*: The host chain is the public bulletin board observed by all nodes. The Privacy Contract records the callable program, variable-to-identifier bindings, and the SS, FHE, and conversion operators. The Address Registry maps each data identifier to a public reference. Invocation and Pick records contain public requests and status only. The chain never stores plaintexts, local SS shares, FHE secret-key shares, MAC-key shares, or preprocessing secrets.

c) *Runtime*: The runtime observes finalized contract events and reads the referenced registry entries. It asks the selected committee to execute the contract operators, coordinates access to the stored values, and submits the accepted output identifier and status. It neither reconstructs a secret value nor replaces the cryptographic protocol run by the committee.

d) *Candidate pool and cryptographic sortition*: The candidate pool lists nodes eligible for an epoch. At the beginning of epoch ℓ , cryptographic sortition selects $\mathcal{S}^\ell = \{P_1, \dots, P_n\}$ and publishes the selection evidence used by the runtime. These parties receive SS uploads, hold live shares, and execute calls until the next handoff [15].

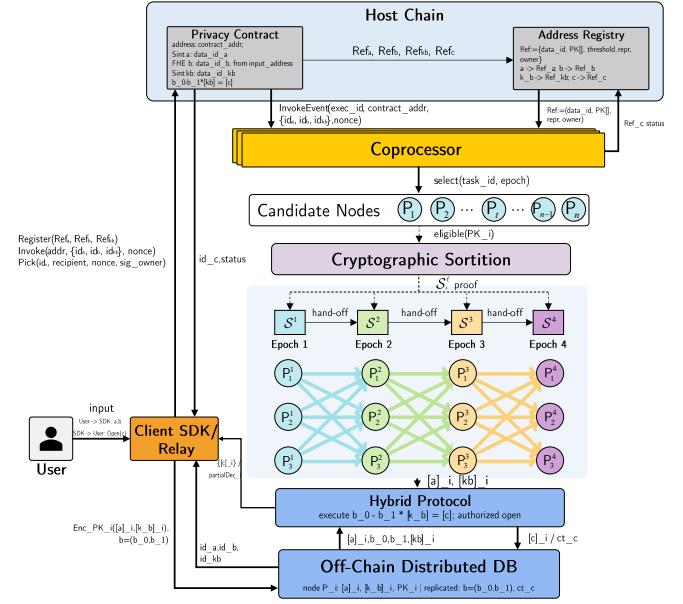


Fig. 1. System components and dataflow. Arrow labels identify the sender, receiver, and transmitted public references or protected values. The host chain stores public contract state and data references; party P_i stores only its own secret shares.

e) *Active committee*: The selected parties execute the contract-defined SS, FHE, and explicit conversion operators. Party P_i reads only its own local shares; replicated FHE ciphertexts are public inputs to the selected operator. The committee stores a protected result and returns only its public identifier and status to the host chain. There is no third persistent “hybrid” representation.

f) *Off-chain storage*: Off-chain storage has two distinct forms. Ciphertext storage may replicate an FHE ciphertext such as $b = (b_0, b_1)$. The protected local store of party P_i contains only records assigned to that party, such as $[a]_i$, $[k]_i$, or $[c]_i$. A single node never stores the complete set of shares needed to reconstruct these values.

C. Runtime

a) *Confidential data representations*: A protected value persists as either an FHE ciphertext under a referenced FHE key or an authenticated secret sharing over $\mathbb{F}_Q$. “Hybrid” means that the contract operator sequence uses both representations; it is not a third value representation.

b) *Contract-defined operator sequence*: The contract states which operators use FHE, which operators use MPC, and where C2S or S2C conversion occurs. It also states the FHE key reference used by an FHE operator.

c) *Operator execution and conversion*: Packed arithmetic is executed with FHE. Comparison is executed with MPC. Multiplication may occur in either domain. Conversion is useful only when the saving from executing an operator in MPC exceeds the cost of converting its inputs and outputs. The benchmark must measure these operator and conversion costs and determine when conversion is useful.

d) *Threshold-FHE key shares*: The threshold FHE secret-key shares have their own data identifier. The active committee holds these shares and hands them over together with live contract-value shares. Here, “KMS-free” means that PPSC eliminates a separate, long-lived KMS committee. Key-share generation, storage, conversion, handoff, and authorized opening remain necessary.

D. Data Structures

The data reference uses only the fields shown in the dataflow above.

- Data reference.

Definition 1 (Data reference). For a confidential value x , the Address Registry stores

$$\text{Ref}_x = (\text{datald}_x, \text{PKSet}_x, \text{threshold}_x, \text{repr}_x, \text{owner}_x), \quad (3)$$

where $\text{repr}_x \in \{\text{SS}, \text{FHE}\}$. For an SS value, PKSet_x identifies its committee members and $\text{threshold}_x = t$ records the privacy threshold and the degree of its Shamir sharing. Any set of at most t shares hides the value, and at least $t + 1$ shares are required for opening. For a replicated FHE ciphertext, PKSet_x identifies its storage or computation nodes and $\text{threshold}_x = 0$ marks an FHE record; the reconstruction threshold belongs to the separately addressed, secret-shared FHE key named by the contract. The owner is checked by a Pick request.

Definition 2 (Contract operator sequence). Each callable function specifies its FHE operators, MPC operators, and explicit C2S or S2C conversions. The sequence also states which key reference is used by an FHE operator.

Definition 3 (Invoke). After uploading its inputs, a user invokes a contract with

$$\text{Invoke}(\text{contractAddr}, \text{inputAddress}, \{\text{datald}_x\}). \quad (4)$$

The invocation contains data identifiers. Plaintexts and secret shares remain outside the invocation. The runtime reads the identifiers and asks the selected committee to execute the operator sequence defined by the contract.

Definition 4 (Pick). To retrieve a result, the user sends

$$\text{Pick}(\text{datald}, \sigma_{\text{owner}}). \quad (5)$$

The signature covers the request tag and datald . The listed committee reads owner from the data reference, verifies the signature under that owner, and returns the shares required for opening.

Each public reference indexes off-chain records under the same identifier. For example, party P_i stores $\text{Local}_i[\text{datald}_a] = [a]_i$ and $\text{Local}_i[\text{datald}_{k_b}] = [k_b]_i$. Ciphertext storage holds $\text{Cipher}[\text{datald}_b] = b$. These mappings are local records, not additional chain state.

E. Running Example

The contract sketch shown in Fig. 1 declares

$$\begin{aligned} \text{address} &: \text{contractAddr}, \\ \text{Sint } a &: \text{datald}_a, \\ \text{Fhe } b &: \text{datald}_b \text{ from inputAddress}, \\ \text{Sint } k_b &: \text{datald}_{k_b}. \end{aligned} \quad (6)$$

The registry contains three independent references:

$$\begin{aligned} \text{Ref}_a &= (\text{datald}_a, \{\text{PK}_i\}, t, \text{SS}, \text{owner}), \\ \text{Ref}_{k_b} &= (\text{datald}_{k_b}, \{\text{PK}_i\}, t, \text{SS}, \text{owner}), \\ \text{Ref}_b &= (\text{datald}_b, \text{PKSet}_b, 0, \text{FHE}, \text{owner}). \end{aligned} \quad (7)$$

The Privacy Contract states that the FHE operator for b uses the secret-shared key k_b . The registry stores the three references above. For the two-component ciphertext used only in this running example, the contract-level conversion relation is

$$[u]_{Q,i}^{\text{poly}} = b_0 - b_1[k_b]_i. \quad (8)$$

This is the share-preserving raw-decryption step shown by the running example; the sign follows its ciphertext convention. The polynomial u is not opened and is not yet the MPC value. Protocol 2 authenticates its coefficients and securely decodes, rounds, and range-checks it to obtain the logical sharing $\langle\langle\mu\rangle\rangle_Q$.

F. End-to-End Dataflow

a) *Contract deployment*: The developer sends the contract program and its callable functions to the host chain and receives contractAddr . The deployed program fixes the variable declarations and operator sequence. Deployment creates no input data identifiers.

b) *SS upload and registration*: The SDK selects a fresh datald_a , creates $[a]_1, \dots, [a]_n$, encrypts each transport message for its intended party $P_i \in \mathcal{S}^\ell$, and sends $(\text{datald}_a, [a]_i)$ to that party. After the parties acknowledge local storage, the owner registers Ref_a . The secret-shared FHE key k_b follows the same flow under datald_{k_b} ; each party receives only its own key share.

c) *FHE upload and registration*: The SDK encrypts the plaintext value under the public key associated by the contract with k_b , stores the resulting ciphertext b under the fresh identifier datald_b , and registers Ref_b . The ciphertext may be replicated. Its storage record contains no secret-key share.

d) *Invocation and output registration*: After upload, the user sends

$$\text{Invoke}(\text{contractAddr}, \text{inputAddress}, \{\text{datald}_a, \text{datald}_b, \text{datald}_{k_b}\}). \quad (9)$$

The chain posts the invocation record. The runtime resolves each reference and uses its PKSet to contact the current execution committee $\mathcal{S}^\ell$. Each selected party loads only its own addressed shares. The ciphertext is supplied as a replicated public input. The committee executes the contract-defined operators and stores a candidate result under a fresh datald_c . Only after

the required protocol checks succeed does the registry accept Ref_c and set the invocation status to Completed; failure leaves $\text{outputId} = \perp$.

e) *Pick and authorized opening*: For an SS result c , the owner posts $\text{Pick}(\text{dataId}_c, \sigma_{\text{owner}})$. A qualified set returns shares of c . For an FHE result encrypted under k_b , the owner obtains the ciphertext and posts $\text{Pick}(\text{dataId}_{k_b}, \sigma_{\text{owner}})$. The runtime resolves Ref_{k_b} , and the listed parties verify its owner before returning at least $t + 1$ key shares. The owner reconstructs k_b and decrypts locally. A request that does not match the recorded owner is rejected.

f) *Epoch transition*: Before handoff, sortition determines the incoming committee $\mathcal{S}^{\ell+1}$. The outgoing committee hands the FHE key shares, live secret-shared values, MAC shares, and remaining conversion or preprocessing state to the incoming committee. After the handoff is verified, the registry replaces PKSet in the SS and key references with the public keys of $\mathcal{S}^{\ell+1}$. Data identifiers, owners, and replicated ciphertexts remain unchanged. The incoming committee then continues execution.

III. HYBRID PROTOCOL DESIGN FOR RUNTIME

runtime support decentr The contract operator sequence specifies every FHE-MPC conversion. This section gives the conversion protocols, fluid committee handoff, and the authenticated protocols used by MPC operators.

A. Fluid Committee Handoff

For epoch ℓ , let

$$\text{State}^\ell = (\text{Key}^\ell, \text{Live}^\ell, \text{MAC}^\ell, \text{Prep}^\ell), \quad (10)$$

where the four components are the FHE key shares, live secret-shared contract values, MAC shares, and remaining conversion or preprocessing state. These objects are transferred together when the committee changes; public data identifiers and replicated ciphertexts do not change.

Protocol 1 $\text{State}^{\ell+1} \leftarrow \Pi_{\text{Handoff}}(\text{State}^\ell)$

Input: The outgoing committee, incoming committee, and State^ℓ .

Output: The incoming committee's shares of the same key, values, MACs, and unused preprocessing values.

Execution:

- The outgoing committee collects the key, live value, MAC, and conversion/preprocessing shares belonging to the current execution state.
- It performs fluid handoff so that the incoming committee receives new shares of the same values without reconstructing them.
- The incoming committee verifies the transferred key, data, and MAC relations and then takes over execution.

B. FHE-MPC Conversion

a) *Conversion parameters*: Fix $R = \mathbb{Z}[X]/(X^N + 1)$ and a conversion level with prime coefficient modulus $Q = q_c$, scale Δ_c , and ciphertext $\text{ct} = (c_0, c_1) \in R_Q^2$. The conversion also fixes the slot layout, fixed-point scale S , and value bound B . Conversion is performed only after the FHE level and scale have been normalized. Encoding, decoding, rounding, and the mapping to $\mathbb{F}_Q$ are performed explicitly during conversion.

Protocol 2 $\langle\langle \mu \rangle\rangle_Q \leftarrow \Pi_{\text{C2S}}(\text{ct}, \text{Ref}_s)$

Public input: The ciphertext reference, key reference, Q , Δ_c , S , and B .

Private state: The committee holds the threshold-FHE key shares and their MAC shares.

Execution:

- Use the data references to obtain the ciphertext and the corresponding key shares.
- Each party computes its share of the decryption intermediate $[u]_{Q,i}^{\text{poly}} = c_0 + c_1[s]_{Q,i}^{\text{poly}}$; the running example uses the sign in (8).
- Without opening u , the committee decodes and rounds it inside MPC, checks its range and wrap-around condition, and obtains $\langle\langle \mu \rangle\rangle_Q$.
- Store the result shares under a fresh data identifier and register the corresponding SS reference.

Privacy rule: Neither u nor the FHE decryption noise is opened.

For the reverse direction, offline preprocessing produces one-time correlated randomness

$$(\langle\langle r \rangle\rangle_Q^{\text{poly}}, \text{ct}_r), \quad (11)$$

where r is uniform in R_Q , ct_r encrypts the same polynomial at the target scale, and the two representations contain the same random value.

Protocol 3 $\text{ct}_\mu \leftarrow \Pi_{\text{S2C}}(\langle\langle \mu \rangle\rangle_Q, \langle\langle r \rangle\rangle_Q^{\text{poly}}, \text{ct}_r)$

Public input: The source data reference, target key reference, Q , Δ_c , S , and B .

Private state: An authenticated sharing $\langle\langle \mu \rangle\rangle_Q$ and the one-time correlated randomness $(\langle\langle r \rangle\rangle_Q^{\text{poly}}, \text{ct}_r)$.

Execution:

- Check the MAC and range of the source sharing.
- Encode the shared value as $\langle\langle m \rangle\rangle_Q^{\text{poly}}$ at the target scale.
- Compute $\langle\langle d \rangle\rangle_Q^{\text{poly}} = \langle\langle m \rangle\rangle_Q^{\text{poly}} - \langle\langle r \rangle\rangle_Q^{\text{poly}}$ and open only $d \in R_Q$ after the MAC check.
- Homomorphically add the public value d to ct_r , store the resulting ciphertext under a fresh data identifier, and register its FHE reference.

Because r is uniform and one-time, $d = m - r \bmod Q$ is independent of m . Also, if decrypting ct_r gives $r + e_r$, then

decrypting the resulting ciphertext gives $m + e_r \pmod{Q}$. This argument requires the two representations of r to contain the same one-time random value. For C2S, correct rounding additionally requires total slot error below $1/2$ and every signed intermediate to lie in $(-Q/2, Q/2)$, unless the conversion performs secure wrap correction. Reusing one mask would reveal differences between source values, so it is forbidden.

C. Authenticated MPC Protocols

D. Supporting Packed Secret Sharing

E. Authenticated MPC Protocols

1) Authentication:

a) *Authenticated SS values:* Every ordinary MPC wire $z \in \mathbb{F}_Q$ is stored as an authenticated sharing

$$\langle\langle z \rangle\rangle_Q = ([z]_Q, [\gamma_z]_Q), \quad \gamma_z = \alpha z, \quad (12)$$

where $\alpha \in \mathbb{F}_Q$ is the committee MAC key, itself secret-shared. The parties keep the value shares, MAC shares, and MAC-key shares off chain. The host chain sees only status updates and output identifiers. When a value remains live across an epoch, handoff transfers both the value sharing and the MAC-tag sharing together with the refreshed sharing of α .

b) *Authenticated opening:* When an authenticated value is opened, the parties also open its MAC information and verify in the next round that

$$[\gamma_z]_Q - z[\alpha]_Q \stackrel{?}{=} 0. \quad (13)$$

A nonzero result rejects the opened value.

c) *Preprocessing:* The offline phase generates the correlated randomness used by the online protocols. Scalar and packed degree reducing consume correlated sharings of the same random value at the two required degrees. Each correlated value is used only once.

2) Offline Preprocessing:

a) *PRSS setup and random sharing:* The preprocessing protocols use the prime field $\mathbb{F}_Q$ fixed for Shamir sharing. Every nonzero evaluation point and every nonzero difference used by the interpolation bases below is invertible. Let $\mathcal{P} = \{P_1, \dots, P_n\}$ and let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_Q \setminus \{0\}$ be distinct public Shamir evaluation points, so $Q > n$. For $0 \leq d < n$, define $\mathcal{H}_d = \{A \subseteq [n] : |A| = n - d\}$. Each $A \in \mathcal{H}_d$ is associated with the public basis

$$u_A^{(d)}(X) = \prod_{j \notin A} \frac{X - \alpha_j}{-\alpha_j}. \quad (14)$$

It has degree d , satisfies $u_A^{(d)}(0) = 1$, and vanishes at α_i for every $i \notin A$. A replicated component known by the parties in A becomes a valid degree- d Shamir component after multiplication by $u_A^{(d)}(X)$.

Following PRSS techniques [16], offline preprocessing gives each holder set an independent common random value. For every coalition C of at most t parties, at least one required random component is unknown to all parties in C .

Protocol 4 $[r_{\text{sh}}]^{(t,Q)} \leftarrow \Pi_{\text{RandShare}}$

Input: $\mathcal{P}$, $\mathbb{F}_Q$, $\{\alpha_i\}_{i=1}^n$, t , and $\mathcal{H}_t$.

Offline randomness: For every $A \in \mathcal{H}_t$, all parties in A hold the same random value r_A .

Output: A degree- t sharing of $r_{\text{sh}} = \sum_{A \in \mathcal{H}_t} r_A$.

Execution:

- Party P_i outputs $[r_{\text{sh}}]_i^{(t,Q)} = \sum_{A \in \mathcal{H}_t} r_A u_A^{(t)}(\alpha_i)$.

The outputs are evaluations of the single polynomial $F_{r_{\text{sh}}}(X) = \sum_{A \in \mathcal{H}_t} r_A u_A^{(t)}(X)$, with $F_{r_{\text{sh}}}(0) = r_{\text{sh}}$ and $F_{r_{\text{sh}}}(\alpha_i) = [r_{\text{sh}}]_i$. For any coalition of at most t parties, the component indexed by its complement is unknown to that coalition.

b) *Correlated scalar pair generation:* Assume $n > 2t$. Degree reducing requires degree- t and degree- $2t$ sharings of the same randomizer ρ . Define $\mathcal{I}_{t,2t} = \{(A, B) \in \mathcal{H}_t \times \mathcal{H}_{2t} : B \subseteq A\}$. The inclusion $B \subseteq A$ ensures that every holder of a high-degree component also holds the corresponding common random value.

Protocol 5 $([\rho]^{(t,Q)}, [\rho]^{(2t,Q)}) \leftarrow \Pi_{\text{PairGen}}$

Input: $\mathcal{P}$, $\mathbb{F}_Q$, $\{\alpha_i\}_{i=1}^n$, t , $\mathcal{H}_t$, $\mathcal{H}_{2t}$, and $\mathcal{I}_{t,2t}$.

Offline randomness: For every $(A, B) \in \mathcal{I}_{t,2t}$, all parties in A hold the same random value $\zeta_{A,B}$.

Output: A correlated pair whose two sharings reconstruct to $\rho = \sum_{(A,B) \in \mathcal{I}_{t,2t}} \zeta_{A,B}$.

Execution:

- Compute the row and column aggregates $\rho_A^{(t)} = \sum_{\substack{B \in \mathcal{H}_{2t} \\ B \subseteq A}} \zeta_{A,B}$ and $\rho_B^{(2t)} = \sum_{\substack{A \in \mathcal{H}_t \\ B \subseteq A}} \zeta_{A,B}$.
- Party P_i outputs $[\rho]_i^{(t,Q)} = \sum_{A \in \mathcal{H}_t} \rho_A^{(t)} u_A^{(t)}(\alpha_i)$ and $[\rho]_i^{(2t,Q)} = \sum_{\substack{B \in \mathcal{H}_{2t} \\ i \in B}} \rho_B^{(2t)} u_B^{(2t)}(\alpha_i)$.

The two outputs are evaluations of degree- t and degree- $2t$ polynomials with the same constant: $F_\rho^{(t)}(X) = \sum_{A \in \mathcal{H}_t} \rho_A^{(t)} u_A^{(t)}(X)$ and $F_\rho^{(2t)}(X) = \sum_{B \in \mathcal{H}_{2t}} \rho_B^{(2t)} u_B^{(2t)}(X)$. They satisfy $F_\rho^{(t)}(0) = \sum_{(A,B) \in \mathcal{I}_{t,2t}} \zeta_{A,B} = F_\rho^{(2t)}(0) = \rho$. The correlated pair is used for degree reducing.

c) *Packed correlated pairs:* Let $\beta_1, \dots, \beta_L \in \mathbb{F}_Q$ be distinct packing points disjoint from the party points, so $Q \geq n + L$. A degree- D polynomial F_x encodes $x = (x_1, \dots, x_L)$ when $F_x(\beta_j) = x_j$ for every $j \in [L]$. The t -private construction uses

$$\begin{aligned} D &= t + L - 1, & \tau &= 2D + 1 - L = 2t + L - 1, \\ L &\leq \left\lfloor \frac{n - 2t + 1}{2} \right\rfloor. \end{aligned} \quad (15)$$

The final bound is equivalent to $n > 2D$ and supplies enough evaluations to reconstruct the degree- $2D$ product sharing.

For $A \in \mathcal{H}_t$ and $j \in [L]$, let $g_{A,j}^{(D)}$ be the unique degree-at-most- D polynomial that equals one at β_j , zero at the other

packing points, and zero at α_i for every $i \notin A$. Similarly, for $B \in \mathcal{H}_\tau$, let $g_{B,j}^{(2D)}$ satisfy the same slot conditions and vanish at every non-holder point of B . Define $\mathcal{I}_{t,\tau}^{\text{pack}} = \{(A, B) \in \mathcal{H}_t \times \mathcal{H}_\tau : B \subseteq A\}$.

Protocol 6 $\Pi_{\text{Pair}}^{\text{pack}}(\langle D, L, Q \rangle, \langle 2D, L, Q \rangle)$

Input: $\mathcal{P}$, $\mathbb{F}_Q$, $\{\alpha_i\}_{i=1}^n$, $\{\beta_j\}_{j=1}^L$, t , L , D , τ , and $\mathcal{I}_{t,\tau}^{\text{pack}}$.

Offline randomness: For each $(A, B) \in \mathcal{I}_{t,\tau}^{\text{pack}}$ and $j \in [L]$, all parties in A hold the same random value $\zeta_{A,B,j}$.

Output: A degree- D /degree- $2D$ packed pair encoding the same vector $\rho = (\rho_1, \dots, \rho_L)$.

Execution:

- Compute the row and column aggregates $\rho_{A,j}^{(D)} = \sum_{\substack{B \in \mathcal{H}_\tau \\ B \subseteq A}} \zeta_{A,B,j}$ and $\rho_{B,j}^{(2D)} = \sum_{\substack{A \in \mathcal{H}_t \\ B \subseteq A}} \zeta_{A,B,j}$.
- Party P_i outputs $\llbracket \rho \rrbracket_i^{(D,L,Q)} = \sum_{j=1}^L \sum_{\substack{A \in \mathcal{H}_t \\ i \in A}} \rho_{A,j}^{(D)} g_{A,j}^{(D)}(\alpha_i)$ and $\llbracket \rho \rrbracket_i^{(2D,L,Q)} = \sum_{j=1}^L \sum_{\substack{B \in \mathcal{H}_\tau \\ i \in B}} \rho_{B,j}^{(2D)} g_{B,j}^{(2D)}(\alpha_i)$.

For every slot j , the low- and high-degree packed sharings encode the same randomizer $\rho_j = \sum_{(A,B) \in \mathcal{I}_{t,\tau}^{\text{pack}}} \zeta_{A,B,j}$ at β_j . As in the scalar case, this pair is used for packed degree reducing.

3) *Online Use of Preprocessing:*

a) *Scalar DN degree reducing:* For degree- t sharings of x and y , the DN multiplication reduction [19] first forms the local product $[z]^{(2t,Q)} = [x]^{(t,Q)}[y]^{(t,Q)}$. With the correlated pair for ρ , parties reconstruct the masked value $[w]^{(2t,Q)} = [z]^{(2t,Q)} + [\rho]^{(2t,Q)}$ and output

$$[z]^{(t,Q)} = w - [\rho]^{(t,Q)}. \quad (16)$$

The pair is used once. Algebraic reconstruction requires $n > 2t$; the parties perform the MAC check when opening the masked value and reject it if the check fails.

b) *Packed degree reducing:* Let F_x and F_y be degree- D packed polynomials. Their product $H_z = F_x F_y$ has degree at most $2D$ and satisfies $H_z(\beta_j) = x_j y_j = z_j$. With the correlated packed pair, the parties reconstruct $H_w(X) = H_z(X) + F_\rho^{(2D)}(X)$, where $w_j = H_w(\beta_j) = z_j + \rho_j$. Let Λ_j be the public Lagrange basis on the packing points and set $W_w(X) = \sum_j w_j \Lambda_j(X)$. Each party outputs

$$F_z^{(D)}(\alpha_i) = W_w(\alpha_i) - \llbracket \rho \rrbracket_i^{(D,L,Q)}. \quad (17)$$

The resulting degree- D polynomial satisfies $F_z^{(D)}(\beta_j) = x_j y_j$ for every slot. The bound $n > 2D$ permits algebraic reconstruction of the masked degree- $2D$ polynomial; the opening is accepted only after its MAC check succeeds.

IV. SECURITY ANALYSIS

The protocol design specifies FHE-MPC conversion and fluid handoff. The concrete malicious-secure conversion and

handoff protocols remain incomplete. This section records the required security conditions. An end-to-end theorem requires complete protocol definitions and proofs.

a) *Operator sequence and leakage:* The contract fixes every operator domain and conversion position. The chain reveals the contract, public data references, and whether an execution succeeds. Protected values remain confidential.

b) *Public-record integrity:* The host chain stores the contract address and data references used by the runtime. A data reference binds a dataId to its owner, SS/FHE representation, committee public keys, and threshold.

c) *Authorized opening:* A Pick request contains the data identifier and the owner's signature. Before returning an SS value share or an FHE key share, each listed party reads the owner from the data reference and verifies the signature. A qualified set of key shares lets the owner reconstruct the full FHE key. The owner can then decrypt every ciphertext under the corresponding public key, so those ciphertexts must share that owner's release policy.

d) *Confidentiality of stored data:* The chain stores no plaintext, local SS fragment, or FHE secret-key share. For an SS value, party P_i receives only its protected local share; replicated storage contains only FHE ciphertexts. Confidentiality depends on the privacy of the selected sharing scheme, the security of the FHE scheme, secure client-to-party transport, and the assumed corruption threshold. The runtime observes identifiers and coordinates execution. It cannot reconstruct a protected value.

e) *Conversion correctness and privacy:* For C2S, the committee computes an authenticated raw-decryption polynomial and runs secure decoding and rounding without opening that polynomial or its noise. Correctness requires the FHE error to be below $1/2$, its range check accepting, and every signed intermediate remaining in $(-Q/2, Q/2)$ unless secure wrap correction is invoked. A proof must also cover malformed ciphertexts and inconsistent key views.

For S2C, offline preprocessing provides an authenticated uniform random value r and an encryption of that same r . Opening $d = m - r \bmod Q$ reveals no information about m for a fresh one-time r , and adding d to the encrypted mask yields an encryption of m with the mask ciphertext's error. This argument fails if the two representations of r do not agree or if r is reused. The final proof must cover generation of the correlated randomness, MAC-checked opening, encoding, and wrap-around.

f) *MAC checks for SS values:* For SS values represented in $\mathbb{F}_Q$, a passing MAC check after an additive deviation requires a consistent tag deviation under the shared MAC key. The concrete protocol must state the algebraic operations and soundness assumptions used by this check. The Shamir and MAC protocols use the Q -parameterized finite-field realization fixed in Section I-B. Arithmetic over a different field or ring requires a separate soundness argument and is not implied by the logical finite-set notation.

g) *Cross-epoch privacy and continuity:* At most t parties may be corrupted in each committee. The handoff must keep

the FHE key, live values, and MAC values secret and transfer their shares to the incoming committee. It must also preserve the relation $\gamma_x = \alpha x$ and transfer the remaining conversion or preprocessing state consistently.

h) Composition obligations.: An end-to-end claim requires complete proofs for authenticated MPC, both conversions, preprocessing, and fluid handoff. Performance claims are deferred until the benchmark is implemented.

V. DISCUSSION AND LIMITATIONS

The conversion layer is intentionally restricted to scalar LWE-style ciphertexts satisfying the exact-scale relation used in the main protocols. Direct conversion of SIMD-packed RLWE or GLWE ciphertexts, slot extraction, and a Galois-ring MPC instantiation are outside the scope of this work. Packed sharing in the protocol section refers only to packed Shamir sharing in the MPC domain. An FHE implementation may use ring ciphertexts internally, but FHE-to-SS must receive, and SS-to-FHE must return, a scalar LWE ciphertext satisfying the stated interface.

The contract-level `euInt64` notation is used only to relate

VI. DISCUSSION AND LIMITATIONS

PPSC uses threshold FHE and bounded fixed-point integers as the logical values used by FHE-MPC conversion. It distinguishes the FHE level modulus q_ℓ and scale Δ_ℓ from the MPC prime-field modulus Q ; at the selected conversion level, $Q = q_c$ only aligns coefficient residues. Secure FHE decoding, encoding, rounding, signed lifting, and wrap checks remain necessary transformations. Packed sharing in the MPC protocols is also distinct from ciphertext-slot packing.

The protocol boxes expose the conversion algebra. Its malicious-secure implementation remains incomplete. The remaining work is to specify secure encoding, decoding, rounding, wrap-around handling, generation of correlated randomness, and checks for modified shares or ciphertexts.

Each ciphertext reference identifies the corresponding FHE key reference. Across handoff, the FHE key and MAC key remain unchanged. The incoming committee receives their shares.

Offline preprocessing generates the correlated randomness used for scalar and packed degree reducing and for conversion. Each correlated random value is used only once. When the committee changes, the remaining preprocessing state is handed to the incoming committee together with the key, live values, and MAC shares.

Removing the standalone KMS service places the FHE key shares in the active committee. After the committee verifies the owner recorded in the key reference, it returns a qualified set of shares. The owner reconstructs the key and decrypts the ciphertext locally.

The concrete malicious-secure implementation and proof of fluid handoff remain future work.

VII. CONCLUSION

PPSC organizes confidential smart-contract execution around one dependency chain. Operator-level FHE/MPC execution requires explicit conversion. The active committee holds the threshold FHE key shares used by conversion together with live value shares, MAC shares, and preprocessing values. Committee rotation transfers all of these values to the incoming committee. Public handles let the host chain refer to these off-chain values. They do not form a third confidential representation.

PPSC specifies this design, conversion algebra, fluid handoff, and authenticated MPC protocols. It does not yet provide the concrete malicious-secure conversion and fluid handoff implementations, a composed security theorem, or benchmark results. Performance claims are deferred until all-FHE with standalone KMS, static hybrid, and dynamic hybrid baselines implement the same contract operators and are measured.

VIII. CONCLUSION

The addressed-state model places FHE secret-key shares, authenticated MPC values, delayed-verification records, and preprocessing material under one epoch-indexed committee lifecycle. The system retains persistent FHE ciphertexts and public references, while dynamic handoff refreshes the committee-local state without changing the FHE key domain. Exact-scale scalar conversion removes wrap-count correction from the main FHE-MPC path, and the PRSS-based preprocessing protocols supply the correlated scalar and packed pairs required by DN degree reduction.

The current protocol body separates the online execution path from external preprocessing and threshold-FHE interfaces. The implementation architecture and planned evaluation are described in Section ??; measured results will be added as the cryptographic and system components are instantiated.

IX. ETHICS CONSIDERATIONS

Confidential smart contracts are dual-use. They protect financial, governance, and identity applications from unnecessary disclosure and can also conceal prohibited activity. A deployment should include explicit access-control rules, audit hooks for authorized disclosure where required by law, and clear limits on the confidentiality guarantee.

REFERENCES

- [1] Zama, “Fhevm: A cross-chain protocol for confidential smart contracts,” technical report, version 3.1, Jun. 2025.
- [2] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, “Zerocash: Decentralized anonymous payments from Bitcoin,” in *Proc. IEEE Symposium on Security and Privacy*, 2014, pp. 459–474.
- [3] S. Bowe, A. Chiesa, M. Green, I. Miers, P. Mishra, and H. Wu, “Zexe: Enabling decentralized private computation,” in *Proc. IEEE Symposium on Security and Privacy*, 2020, pp. 820–837.
- [4] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou, “Hawk: The blockchain model of cryptography and privacy-preserving smart contracts,” in *Proc. IEEE Symposium on Security and Privacy*, 2016, pp. 839–858.
- [5] G. Kaptchuk, M. Green, and I. Miers, “Giving state to the stateless: Augmenting trustworthy computation with ledgers,” in *Proc. Network and Distributed System Security Symposium*, 2019.

- [6] A. Banerjee, M. Clear, and H. Tewari, “zkHawk: Practical private smart contracts from MPC-based Hawk,” in *Proc. 3rd Conference on Blockchain Research & Applications for Innovative Networks and Services*, 2021, pp. 245–248.
- [7] A. Banerjee and H. Tewari, “Multiverse of HawkNess: A universally-composable MPC-based Hawk variant,” *Cryptography*, vol. 6, no. 3, Art. 39, 2022.
- [8] C. Baum, J. H.-Y. Chiang, B. David, and T. K. Frederiksen, “Eagle: Efficient privacy preserving smart contracts,” in *Proc. Financial Cryptography and Data Security*, 2023.
- [9] B. Bünz, S. Agrawal, M. Zamani, and D. Boneh, “Zether: Towards privacy in a smart contract world,” in *Proc. Financial Cryptography and Data Security*, 2020, pp. 423–443.
- [10] S. Steffen, B. Bichsel, M. Gersbach, N. Melchior, P. Tsankov, and M. Vechev, “zkay: Specifying and enforcing data privacy in smart contracts,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1759–1776.
- [11] R. Solomon, R. Weber, and G. Almashaqbeh, “smartFHE: Privacy-preserving smart contracts from fully homomorphic encryption,” in *Proc. IEEE European Symposium on Security and Privacy*, 2023, pp. 309–331.
- [12] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, and A. Sahai, “Threshold cryptosystems from threshold fully homomorphic encryption,” in *Advances in Cryptology—CRYPTO*, 2018, pp. 565–596.
- [13] M. Dahl, D. Demmler, S. El Kazdadi, A. Meyre, J.-B. Orfila, D. Rotaru, N. P. Smart, S. Tap, and M. Walter, “Noah’s Ark: Efficient Threshold-FHE Using Noise Flooding,” *Cryptology ePrint Archive*, Report 2023/815, 2023.
- [14] A. Herzberg, S. Jarecki, H. Krawczyk, and M. Yung, “Proactive secret sharing or: How to cope with perpetual leakage,” in *Advances in Cryptology—CRYPTO*, 1995, pp. 339–352.
- [15] A. R. Choudhuri, A. Goel, M. Green, A. Jain, and G. Kaptchuk, “Fluid MPC: Secure multiparty computation with dynamic participants,” in *Advances in Cryptology—CRYPTO*, Part II, 2021, pp. 94–123.
- [16] R. Cramer, I. Damgård, and Y. Ishai, “Share conversion, pseudorandom secret-sharing and applications to secure computation,” in *Theory of Cryptography Conference—TCC*, LNCS 3378, 2005, pp. 342–362.
- [17] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [18] M. K. Franklin and M. Yung, “Communication complexity of secure computation,” in *Proc. 24th Annual ACM Symposium on Theory of Computing*, 1992, pp. 699–710.
- [19] I. Damgård and J. B. Nielsen, “Scalable and unconditionally secure multiparty computation,” in *Advances in Cryptology—CRYPTO*, LNCS 4622, 2007, pp. 572–590.
- [20] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, “Multiparty computation from somewhat homomorphic encryption,” in *Advances in Cryptology—CRYPTO*, LNCS 7417, 2012, pp. 643–662.
- [21] V. Goyal and Y. Song, “Malicious security comes free in honest-majority MPC,” *Cryptology ePrint Archive*, Report 2020/134, 2020.